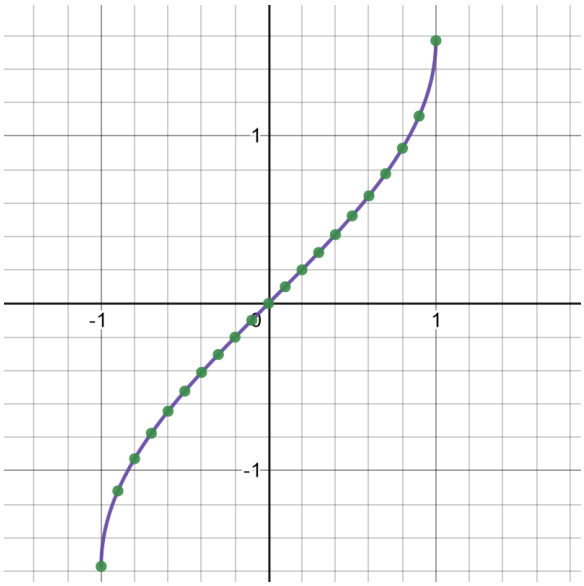
Darren Lee
danalee@ucsc.edu
CSE 13S Spring 2021

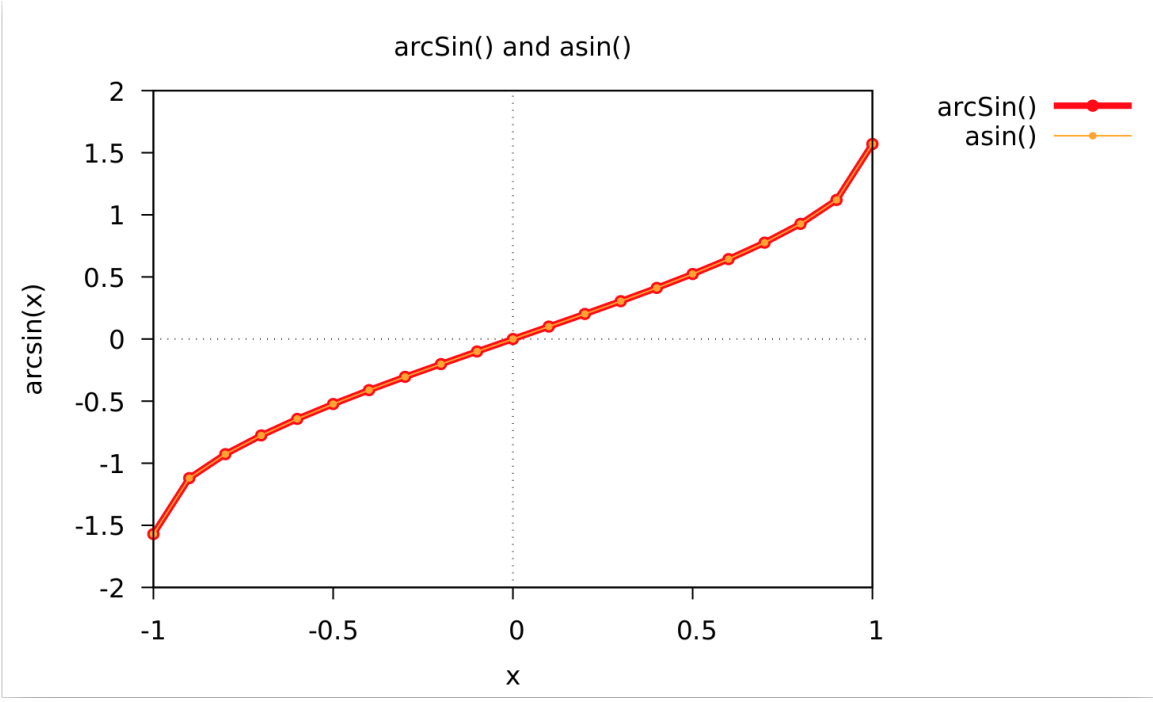
Writeup

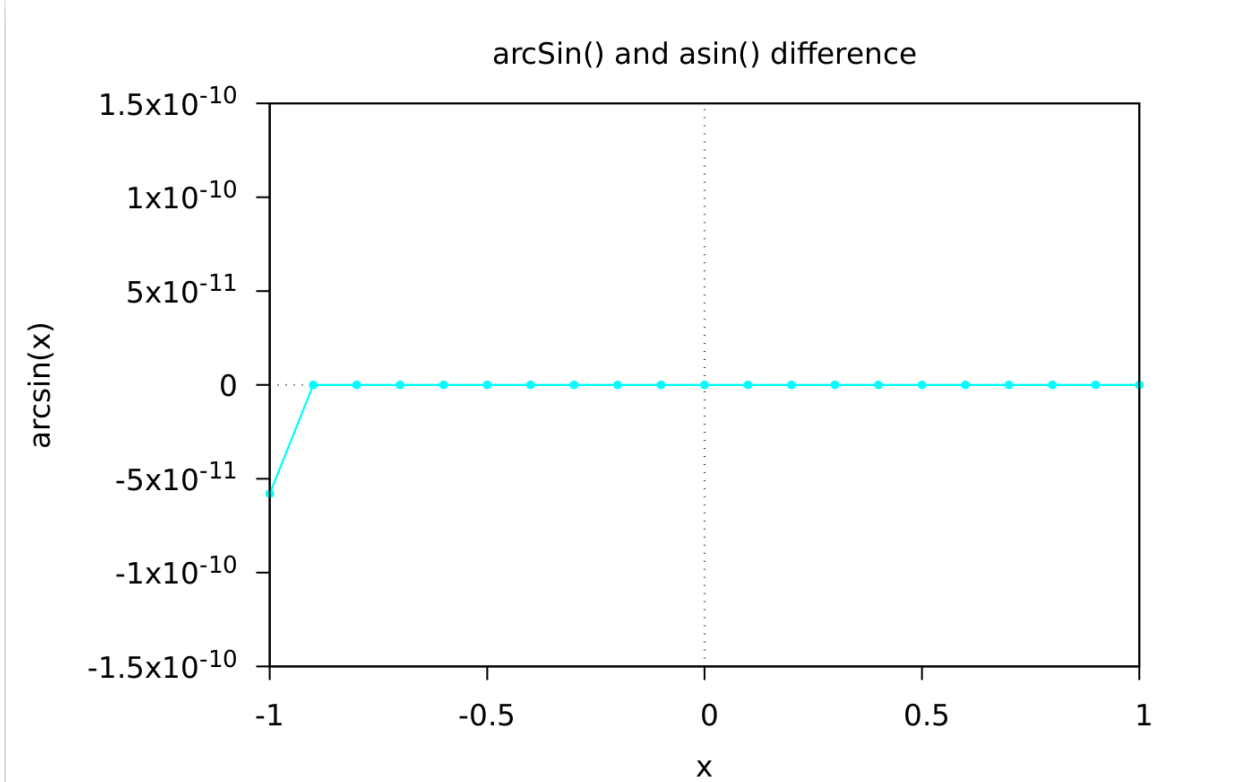
My gnuplot graphs were created with code from Eugene's 4/15/21 lab section.

arcSin graphs
Desmos:



gnuplot:





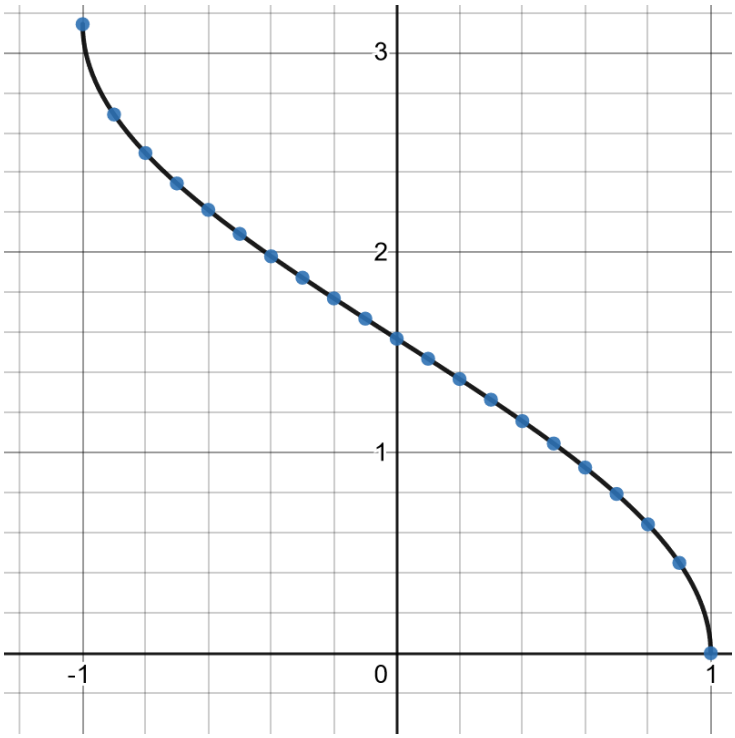
Analysis

I used inverses and Newton's method to approximate arcsin. The largest difference between my arcSin() function and the <math.h> library was for x=-1. The difference was -0.00000000005821 or -5.821×10^{-11} and was larger than the other differences because it is near a steep part of the inverse sine function graph where the y-values change most quickly and the tangent line is nearly vertical. I used Newton's method and a trigonometric identity,

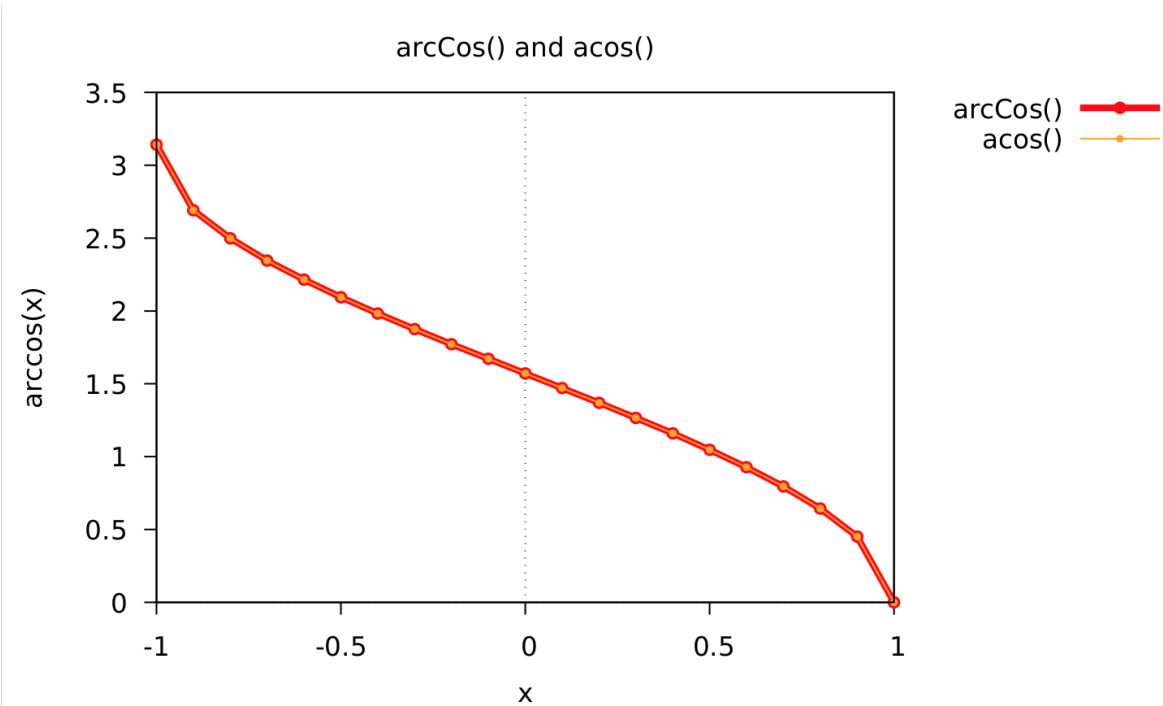
$\cos(\arcsin(x)) = \sqrt{1-x^2}$ and $arcsin(x) = arccos(\sqrt{(1-x^2)})$ for 0<=x<=1, to calculate arcSin when x is near -1 and 1 because Newton's method alone produced too large of an error for -1 and 1. Using the trigonometric identity allows me to calculate arcCos(0), which is not as prone to error, instead. However, the output for x=-1 has a greater degree of inaccuracy than x=1 because I had to correct the guess value for arcSin for x=-1 by subtracting π (guess - π). I did not need to adjust by -π for x=1, so the value is more accurate (0.00000000000000). Since π in the math.h library (M_PI) is an approximation for π, it likely caused my arcSin value for x=-1 to lose accuracy.

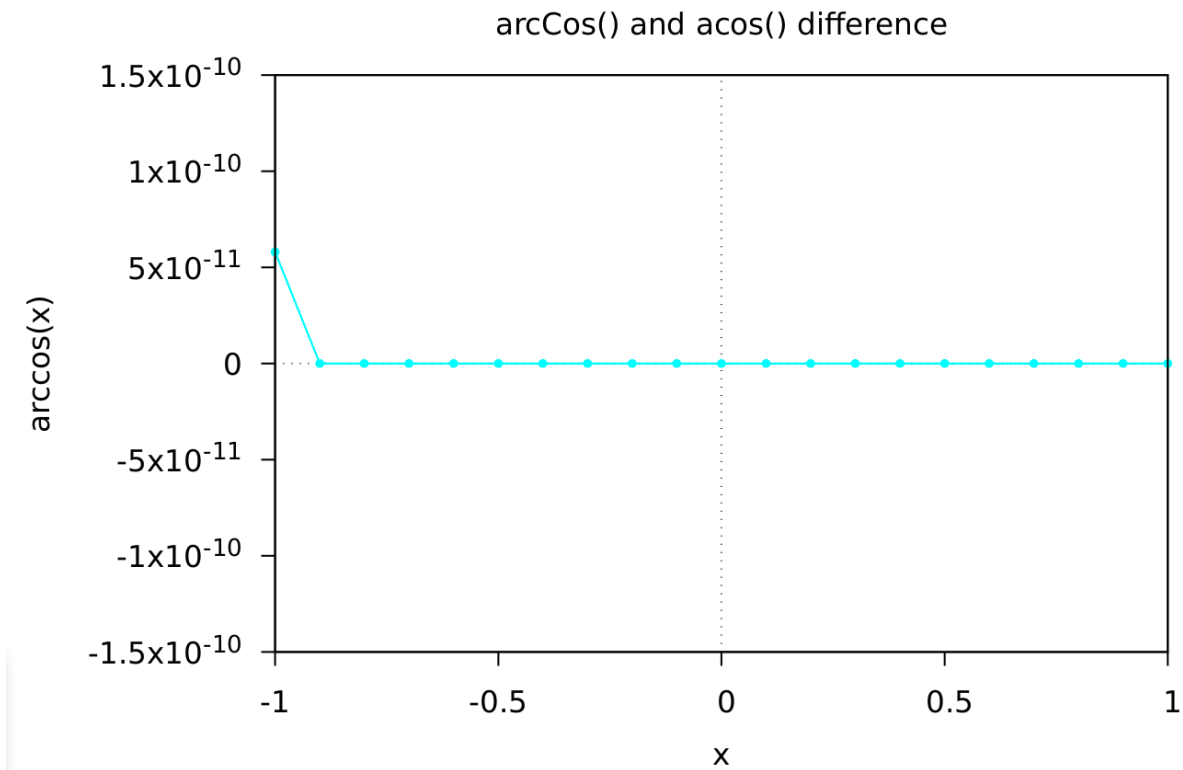
arcCos graphs

Desmos:



gnuplot:





Analysis

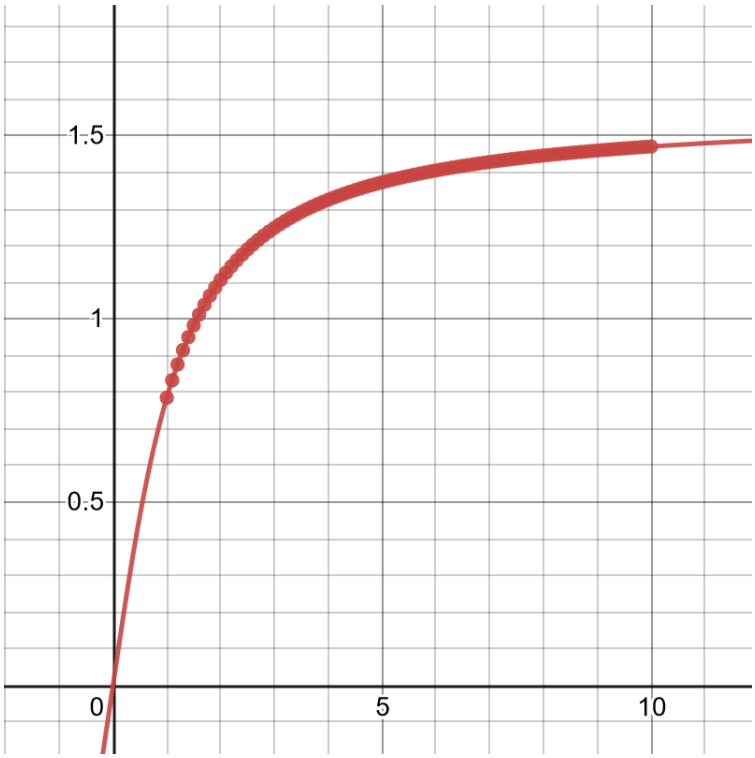
My arcCos() function relies on my arcSin() function through the trigonometric identity arccos(x)=π / 2 - arcsin(x), so the arcCos() function similarly has its largest difference at x=-1. The difference is 0.00000000005821 or 5.821×10^{-11}. Due to my arcCos() functions reliance on my arcSin() function, it also relies on Newton's method. Newton's method is accurate for values that are not near -1 and 1, but at -1 and 1 the tangent lines are steep and have slopes close to infinity. Since Newton's method relies on tangent lines and uses the following equation:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

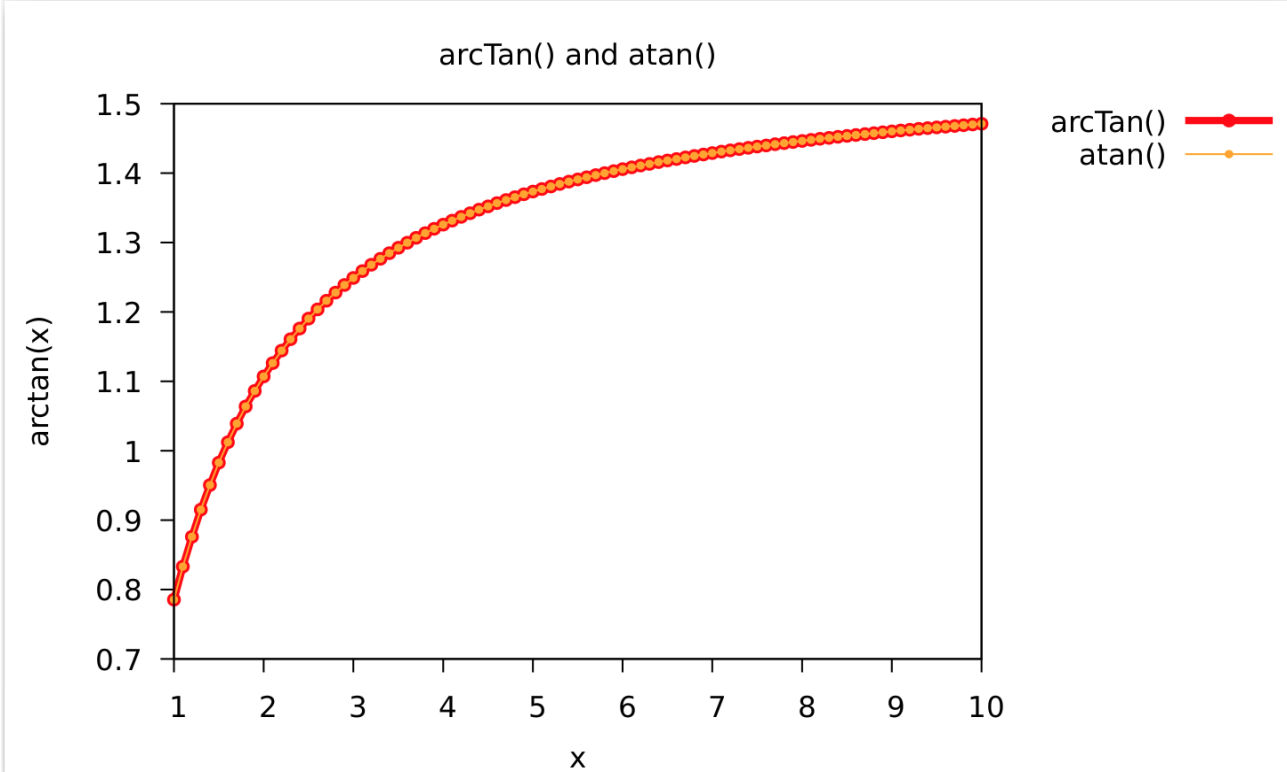
, a derivative close to infinity would lead to almost no change in the terms and make it more difficult to reach an accurate approximation for arcCos.

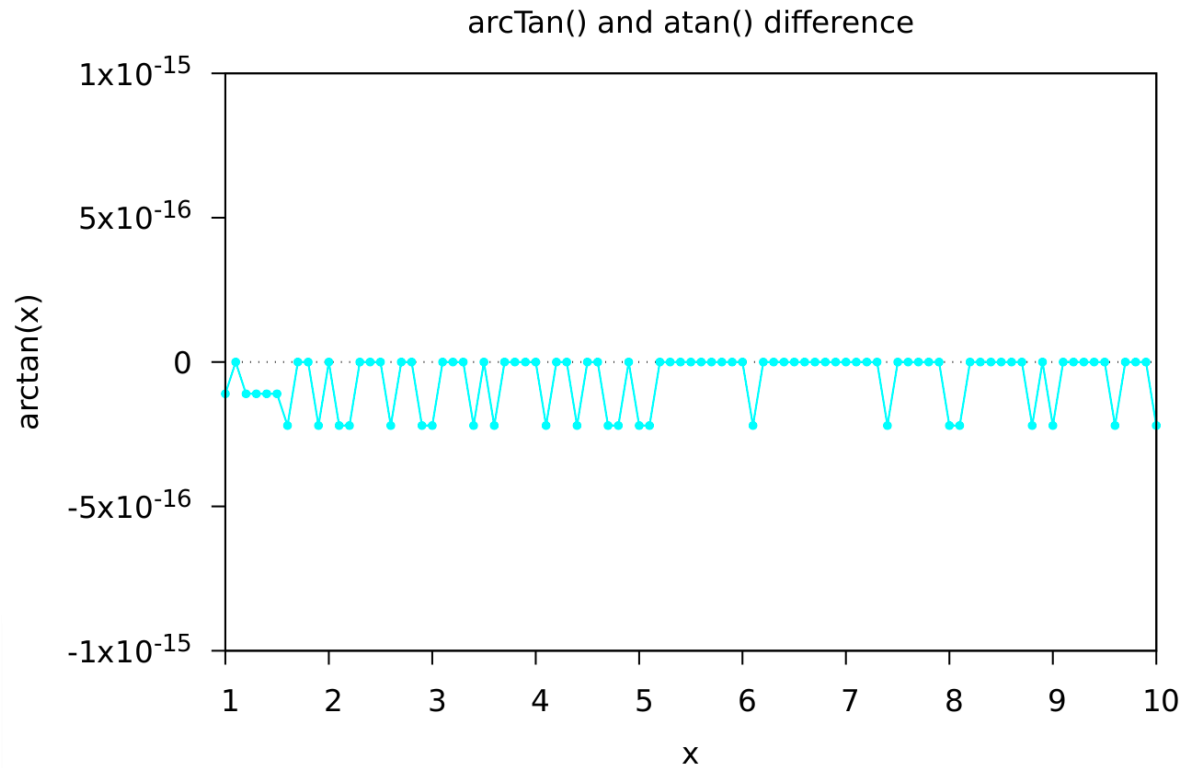
arcTan graphs

Desmos:



gnuplot:

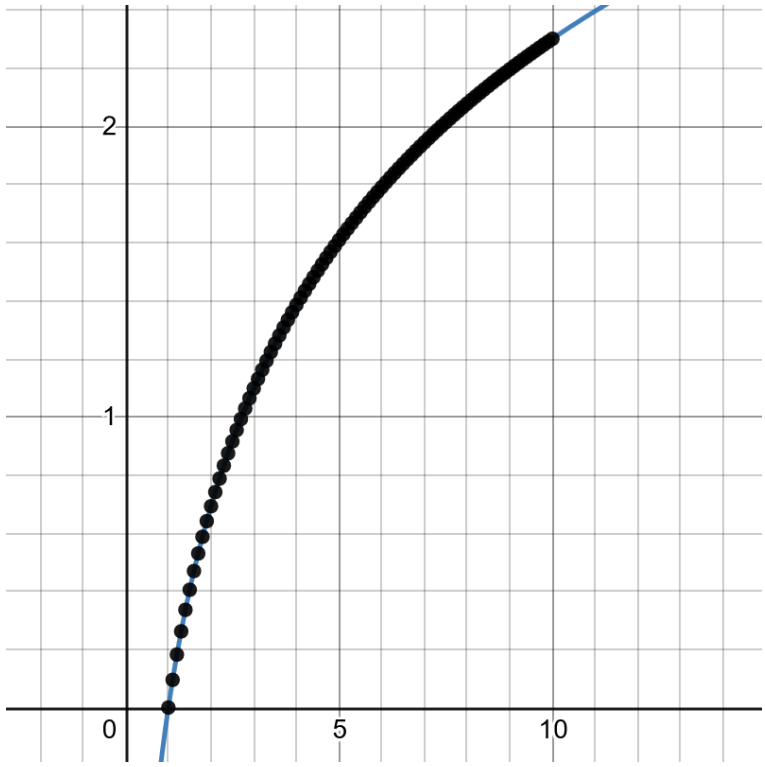




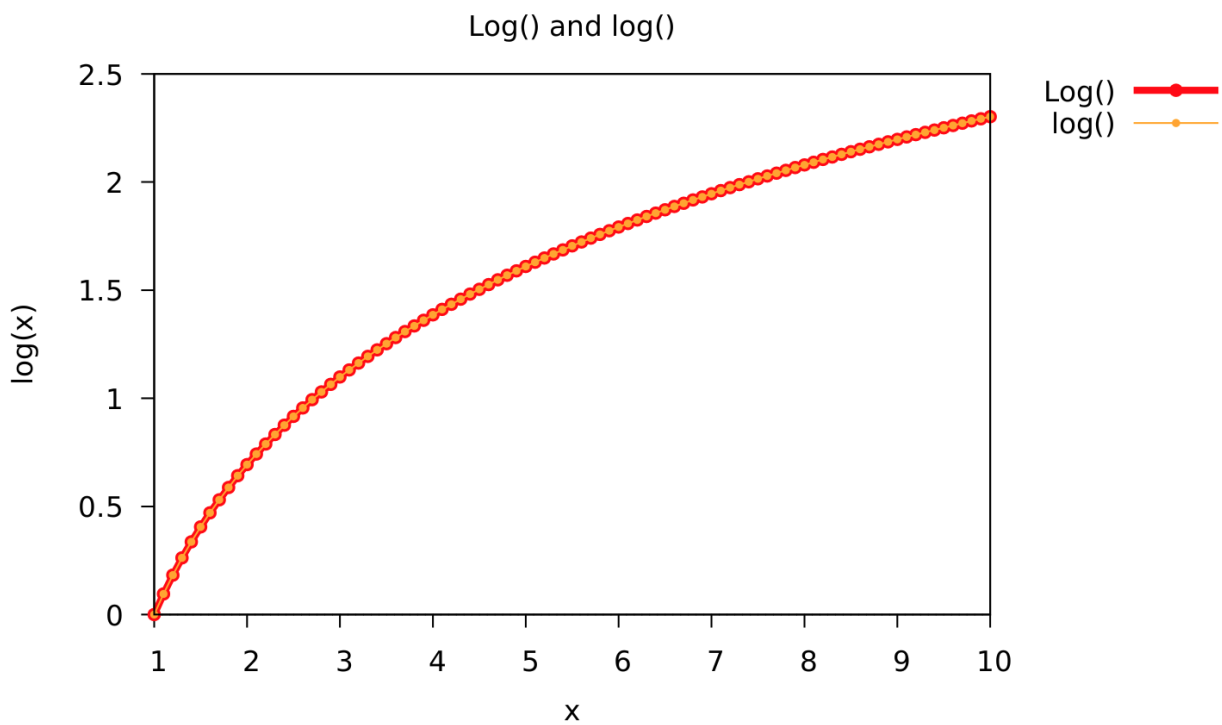
Analysis

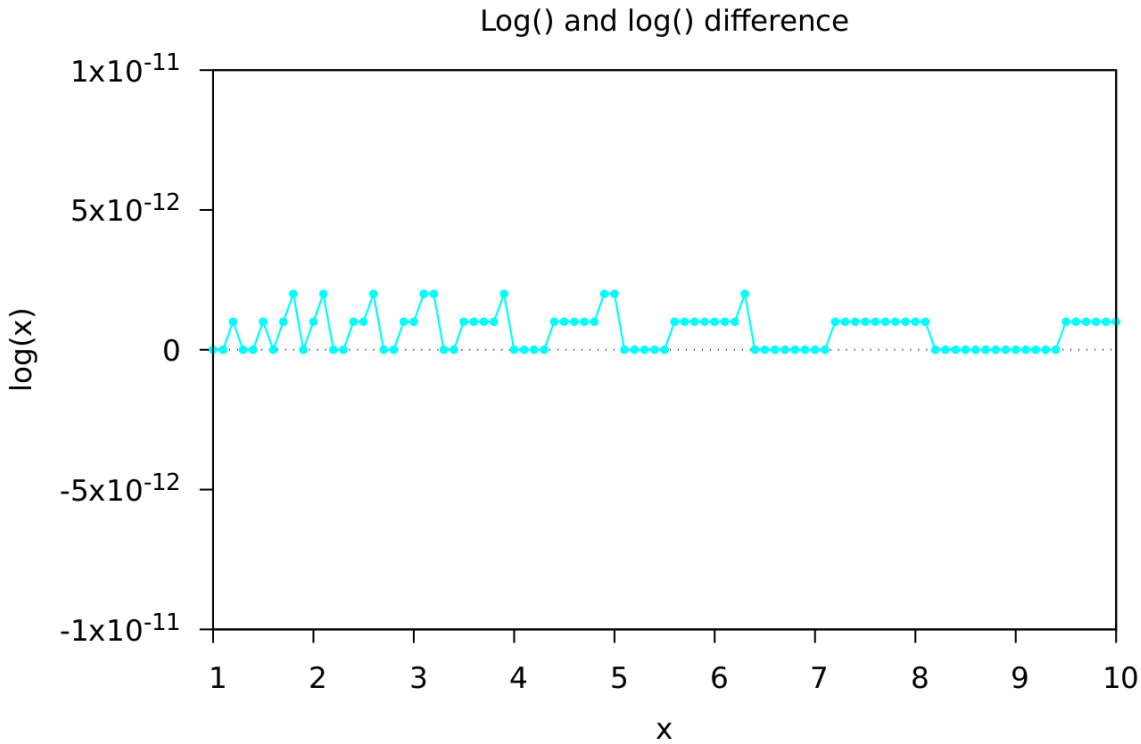
My arcTan() function is very close to the math.h library values with minimal differences. I used the trigonometric identity $\arctan(x) = \arccos\left(\frac{1}{\sqrt{x^2+1}}\right), \quad x > 0.$ to implement my arcTan() function. Since I used Newton's method to implement arcSin() and arcCos() uses arcSin(), arcTan() also uses Newton's method. Since there are no horizontal or vertical tangents in the test range of [1,10] for arctan, Newton's method is very accurate for this function. However, there are still differences from the <math.h> library because Newton's method only continues making successive guesses until the difference between guesses is within the supplied epsilon value.

Log graph
Desmos:



gnuplot:





Analysis

I also used Newton's method for my Log() function. This implementation works well and matches the math.h library fairly closely because there are no horizontal or vertical tangents in the range [1,10], so the approximations are able to converge to the roots. However, there are still differences from the <math.h> library because Newton's method only continues making successive guesses until the difference between guesses is within the supplied epsilon value.